

Patient Portal - Detailed Technical Requirements

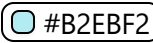
Overview

The Patient Portal provides patients with a comprehensive dashboard to manage their healthcare needs, including appointments, messages, billing, lab results, and device monitoring.

1. Navigation & Layout

1.1 Top Navigation Bar

Visual Requirements:

- Background: Light blue/teal gradient (( #E0F7FA) to ( #B2EBF2))
- Logo: "Sarhi" text in dark blue on left side
- Navigation tabs: Patient, Clinician, Referrals, Billing, Inventory, Health, Owner, AI Agents, Voice Agents, Config
- Active tab: White background with rounded top corners
- Inactive tabs: Transparent background, gray text
- Layout: Fixed header, height 64px

Implementation:

```
typescript

interface NavigationTab {
  id: string;
  label: string;
  href: string;
  isActive: boolean;
}
```

Acceptance Criteria:

- Current portal tab ("Patient") must have white background and dark text
- Other tabs must be clickable and navigate to respective portals
- Header must remain fixed during scroll
- Logo must link to home dashboard

1.2 Left Sidebar Navigation

Visual Requirements:

- Width: 240px on desktop, collapsible on mobile
- Background: White with subtle shadow
- Active item: Purple background ( #9C27B0) with white text and rounded corners
- Inactive items: Gray text with subtle hover state
- Icons: 16px, left-aligned with 12px margin

Navigation Items (in order):

1.  Dashboard (active)
2.  Profile
3.  Medications
4.  Lab Results
5.  Messages
6.  Appointments
7.  Billing

Implementation:

```
typescript

interface SidebarItem {
  id: string;
  label: string;
  icon: React.ComponentType;
  href: string;
  isActive: boolean;
  badge?: number; // For unread counts
}
```

Acceptance Criteria:

- "Dashboard" must be highlighted as active page
- Each item must have appropriate icon and navigate to correct page
- Hover states must show light gray background for inactive items
- Mobile: Sidebar must collapse to hamburger menu below 768px

2. Dashboard Content Area

2.1 Welcome Header

Visual Requirements:

- Text: "Welcome back, Jane!" in large, bold font (24px)
- Color: Dark gray (● #333333)
- Margin: 24px bottom

Implementation:

```
typescript

interface WelcomeHeader {
  patientName: string;
  lastLoginDate?: string;
}
```

Acceptance Criteria:

- Patient name must be dynamically populated from user data
- Text must be responsive (smaller on mobile)

2.2 Summary Cards Row

Card Layout Specifications:

- **Grid:** 3 columns on desktop, 1 column on mobile
- **Gap:** 24px between cards
- **Card Height:** 120px minimum
- **Border Radius:** 8px
- **Shadow:** subtle (0 2px 4px rgba(0,0,0,0.1))

2.2.1 Upcoming Appointments Card

Visual Requirements:

- Background: White
- Header: "Upcoming Appointments" (16px, semibold)
- Main Number: "2" (32px, bold, primary color)
- Subtitle: "Your next appointment is in 3 days" (14px, gray)
- Button: "View Appointments" (full width, blue background, white text)

Data Requirements:

typescript

```
interface AppointmentSummary {  
  totalUpcoming: number;  
  nextAppointmentDate: string;  
  nextAppointmentProvider: string;  
  daysUntilNext: number;  
}
```

Acceptance Criteria:

- Must show accurate count of future appointments
- Must calculate days until next appointment correctly
- Button must navigate to /patient/appointments
- If no appointments: Show "0" and "Schedule your next visit"

2.2.2 Unread Messages Card

Visual Requirements:

- Background: White
- Header: "Unread Messages" (16px, semibold)
- Main Number: "1" (32px, bold, primary color)
- Subtitle: "From Dr. Emily Carter" (14px, gray)
- Button: "Go to Inbox" (full width, blue background, white text)

Data Requirements:

typescript

```
interface MessageSummary {  
  unreadCount: number;  
  latestMessageFrom: string;  
  latestMessageDate: string;  
  hasUrgentMessages: boolean;  
}
```

Acceptance Criteria:

- Must show accurate unread message count
- Must display sender of most recent unread message
- If urgent messages exist, show red badge or indicator
- Button must navigate to /patient/messages
- If no unread messages: Show "0" and "All caught up!"

2.2.3 Outstanding Bills Card

Visual Requirements:

- Background: White
- Header: "Outstanding Bills" (16px, semibold)
- Main Number: "\$400.00" (32px, bold, primary color)
- Subtitle: "2 bills are currently due for payment" (14px, gray)
- Button: "Pay Bills" (full width, blue background, white text)

Data Requirements:

```
typescript

interface BillingSummary {
  totalOutstanding: number;
  billCount: number;
  oldestBillDate: string;
  hasOverdueBills: boolean;
}
```

Acceptance Criteria:

- Must sum all outstanding bill amounts accurately
- Must count individual bills correctly
- If overdue bills exist, show red indicator
- Button must navigate to /patient/billing
- Amount must be formatted as currency with \$ symbol

2.3 At-Home Devices Section

Visual Requirements:

- **Section Header:** "  My At-Home Devices" (18px, semibold)

- **Subtitle:** "Manage devices checked out to you for at-home monitoring." (14px, gray)
- **Card Background:** Light blue (#F3F4F6)
- **Device Icon:** ⚠ warning triangle
- **Device Name:** "Smart Scale" (16px, semibold)
- **Due Date:** "Return by September 30th, 2025 (in 15 days)" (14px)
- **Description:** Patient education text about device usage

Data Requirements:

typescript

```
interface PatientDevice {
  id: string;
  deviceName: string;
  deviceType: 'scale' | 'monitor' | 'glucometer' | 'oximeter';
  assignedDate: string;
  dueDate: string;
  instructions: string;
  isOverdue: boolean;
  remindersSent: number;
}
```

Implementation Rules:

1. Device Display Logic:

- Show only devices assigned to current patient
- Display most recently assigned device first
- Hide section if no devices assigned

2. Due Date Calculation:

typescript

```
const daysUntilDue = differenceInDays(dueDate, new Date());
const dueDateText = `Return by ${format(dueDate, 'MMMM do, yyyy')} (${
  daysUntilDue > 0 ? `in ${daysUntilDue} days` :
  daysUntilDue === 0 ? 'due today' :
  `${Math.abs(daysUntilDue)} days overdue`
})`;
```

3. Visual States:

- **Normal:** Blue background, regular text

- **Due Soon** (≤ 3 days): Yellow background, bold text
- **Overdue**: Red background, white text, urgent messaging

Acceptance Criteria:

- Must only display if patient has assigned devices
- Must calculate due date correctly with human-friendly text
- Must show appropriate urgency based on due date
- Device instructions must be clear and actionable
- Must handle multiple devices (show all assigned devices)

2.4 Quick Actions Section

Visual Requirements:

- **Section Header**: "Quick Actions" (18px, semibold)
- **Subtitle**: "What would you like to do today?" (14px, gray)
- **Button Grid**: 2x2 on desktop, 1 column on mobile
- **Button Style**: White background, gray border, 12px padding
- **Button Text**: 14px, dark gray
- **Button Height**: 48px minimum

Action Buttons (in order):

1. Book a New Appointment

- Navigate to: /patient/appointments/new
- Icon:  calendar

2. View Lab Results

- Navigate to: /patient/lab-results
- Icon:  test tube

3. Update Your Profile

- Navigate to: /patient/profile
- Icon:  person

4. Contact the Office

- Action: Open contact modal or navigate to contact page
- Icon:  phone

Implementation:

```
typescript  
  
interface QuickAction {  
  id: string;  
  label: string;  
  icon: string;  
  href?: string;  
  onClick?: () => void;  
  disabled?: boolean;  
}
```

Acceptance Criteria:

- All buttons must be clearly clickable with proper hover states
- Navigation must work correctly for each action
- Buttons must be accessible via keyboard navigation
- Mobile layout must stack buttons vertically
- Icons must be consistent with overall design system

3. Responsive Design Requirements

3.1 Breakpoints

- **Mobile:** 320px - 767px
- **Tablet:** 768px - 1023px
- **Desktop:** 1024px+

3.2 Mobile Adaptations (≤ 767 px)

1. Navigation:

- Top nav becomes horizontal scroll if needed
- Sidebar collapses to hamburger menu
- Menu opens as full-screen overlay

2. Content Layout:

- Summary cards stack vertically (1 column)
- Quick action buttons stack vertically
- Reduce padding/margins by 50%

3. Typography:

- Welcome header: 20px (down from 24px)
- Card numbers: 28px (down from 32px)
- Maintain readability at smallest screen sizes

3.3 Tablet Adaptations (768px-1023px)

1. Layout:

- Summary cards: 2 columns, 3rd card on new row
- Sidebar remains visible but narrower (200px)
- Quick actions: 2x2 grid maintained

4. State Management & Data Flow

4.1 Component State

typescript

```
interface PatientDashboardState {  
  isLoading: boolean;  
  patient: Patient;  
  appointmentSummary: AppointmentSummary;  
  messageSummary: MessageSummary;  
  billingSummary: BillingSummary;  
  assignedDevices: PatientDevice[];  
  error: string | null;  
}
```

4.2 Data Loading

1. On Mount:

- Load patient profile from localStorage or API
- Load appointment summary
- Load message counts
- Load billing summary
- Load assigned devices

2. Real-time Updates:

- Listen for new messages (update unread count)
- Listen for appointment changes

- Listen for billing updates
- Listen for device assignment changes

4.3 Error Handling

1. Data Load Failures:

- Show error message in place of failed section
- Provide retry button
- Don't block other sections from loading

2. Navigation Failures:

- Show toast notification
- Log error for debugging
- Fallback to safe navigation

5. Performance Requirements

5.1 Loading Standards

- **Initial page load:** <800ms
- **Data fetching:** <300ms per section
- **Navigation transitions:** <200ms
- **Interactive elements:** <100ms response time

5.2 Optimization Strategies

1. **Code Splitting:** Lazy load non-critical components
2. **Data Prefetching:** Preload likely next pages
3. **Image Optimization:** Compress and lazy load images
4. **Caching:** Cache API responses appropriately

6. Accessibility Requirements

6.1 WCAG 2.1 AA Compliance

1. **Color Contrast:** Minimum 4.5:1 for normal text
2. **Keyboard Navigation:** All interactive elements accessible
3. **Screen Readers:** Proper ARIA labels and landmarks
4. **Focus Management:** Clear focus indicators

6.2 Specific Implementations

```
html

<!-- Navigation -->
<nav role="navigation" aria-label="Patient portal navigation">
  <ul role="menubar">
    <li role="menuitem" aria-current="page">Dashboard</li>
  </ul>
</nav>

<!-- Summary Cards -->
<section aria-labelledby="appointments-heading">
  <h2 id="appointments-heading">Upcoming Appointments</h2>
  <span aria-live="polite">{appointmentCount} appointments</span>
</section>

<!-- Quick Actions -->
<section aria-labelledby="quick-actions-heading">
  <h2 id="quick-actions-heading">Quick Actions</h2>
  <button aria-describedby="book-appointment-desc">
    Book a New Appointment
  </button>
</section>
```

7. Testing Requirements

7.1 Unit Tests

- Component rendering with mock data
- Data calculation functions (due dates, counts)
- Navigation link accuracy
- Responsive layout at different breakpoints

7.2 Integration Tests

- Data loading from localStorage
- Real-time update handling
- Cross-portal navigation
- Error state handling

7.3 E2E Tests

- Complete dashboard load workflow
- Navigation to all linked pages
- Mobile responsive behavior
- Accessibility compliance

Patient Portal Sub-Features - Detailed Requirements

8. Patient Billing Page (/patient/billing)

8.1 Page Layout & Navigation

Visual Requirements:

- Page title: "Billing & Payments" (24px, bold)
- Breadcrumb: Dashboard > Billing
- Two-tab interface: "Outstanding Bills" and "Payment History"
- Tab styling: Same as main navigation tabs

8.2 Outstanding Bills Tab

Table Structure:

```
typescript
interface OutstandingBill {
  id: string;
  statementDate: string;
  serviceDate: string;
  description: string;
  amount: number;
  dueDate: string;
  status: 'due' | 'overdue' | 'payment_plan';
  isOverdue: boolean;
  daysPastDue?: number;
}
```

Visual Requirements:

- **Table Headers:** Date of Service | Description | Amount Due | Due Date | Status | Actions
- **Row Styling:**
 - Normal: White background

- Overdue: Light red background ( #FEF2F2)
- Payment plan: Light blue background ( #F0F9FF)
- **Amount Formatting:** Bold, \$ symbol, right-aligned
- **Status Badges:**
 - Due: Blue badge
 - Overdue: Red badge with days past due
 - Payment Plan: Green badge

Pay Now Button Implementation:

```
typescript
interface PaymentAction {
  billId: string;
  amount: number;
  minimumPayment?: number;
  allowPartialPayment: boolean;
  paymentMethods: ('card' | 'bank' | 'check')[];
}

const handlePayNow = (bill: OutstandingBill) => {
  // Open payment modal with bill details
  setSelectedBill(bill);
  setPaymentModalOpen(true);
};
```

Acceptance Criteria:

- Must show only bills with status 'due' or 'overdue'
- Must calculate days past due correctly for overdue bills
- "Pay Now" button must open payment processing modal
- Must handle partial payments if enabled
- Must show payment plan options if available
- Empty state: "No outstanding bills. You're all caught up!"

8.3 Payment History Tab

Table Structure:

typescript

```
interface PaymentHistoryItem {  
  id: string;  
  paymentDate: string;  
  serviceDate: string;  
  description: string;  
  amountPaid: number;  
  paymentMethod: 'card' | 'bank' | 'check' | 'cash';  
  confirmationNumber: string;  
  receiptUrl?: string;  
}
```

Visual Requirements:

- **Table Headers:** Payment Date | Service Date | Description | Amount Paid | Method | Confirmation | Receipt
- **Payment Method Icons:**
 - Card: 
 - Bank: 
 - Check: 
 - Cash: 
- **Receipt Action:** "View Receipt" button or download icon

Acceptance Criteria:

- Must show payments in reverse chronological order (newest first)
- Must provide receipt download/view functionality
- Must show payment method clearly
- Search/filter functionality for date ranges
- Pagination for large payment histories

8.4 Payment Processing Modal

Modal Structure:

typescript

```
interface PaymentForm {  
  billId: string;  
  amount: number;  
  paymentMethod: 'card' | 'bank';  
  cardDetails?: {  
    number: string;  
    expiryMonth: string;  
    expiryYear: string;  
    cvv: string;  
    name: string;  
  };  
  bankDetails?: {  
    routingNumber: string;  
    accountNumber: string;  
    accountType: 'checking' | 'savings';  
  };  
  billingAddress: Address;  
  savePaymentMethod: boolean;  
}
```

Security Requirements:

- Never store full credit card numbers
- Use payment tokenization
- PCI DSS compliance for card processing
- SSL encryption for all payment data

9. Patient Lab Results Page (/patient/lab-results)

9.1 Page Layout

Visual Requirements:

- Page title: "Lab Results" (24px, bold)
- Filter controls: Date range, test type, status
- Results grouped by date (newest first)
- Clear visual distinction between normal and abnormal results

9.2 Lab Result Card Design

Normal Results Card:

typescript

```
interface NormalLabResult {  
  id: string;  
  testName: string;  
  testDate: string;  
  orderingProvider: string;  
  status: 'normal';  
  canViewDetails: true;  
  summary: string;  
}
```

Visual Styling:

- Background: Light green ( #F0FDF4)
- Border: Green accent ( #22C55E)
- Status icon:  checkmark
- "View Details" button: Green background

Abnormal Results Card:

typescript

```
interface AbnormalLabResult {  
  id: string;  
  testName: string;  
  testDate: string;  
  orderingProvider: string;  
  status: 'abnormal' | 'critical';  
  canViewDetails: false;  
  contactMessage: string;  
  urgencyLevel: 'high' | 'critical';  
}
```

Visual Styling:

- Background: Light red ( #FEF2F2)
- Border: Red accent ( #EF4444)
- Status icon:  warning triangle
- "Contact Office" button: Red background
- Critical results: Animated pulse border

9.3 Lab Result Detail Modal (Normal Results Only)

Modal Content:

```
typescript
interface LabResultDetails {
  testName: string;
  testDate: string;
  results: {
    testComponent: string;
    value: string;
    unit: string;
    referenceRange: string;
    status: 'normal' | 'low' | 'high';
  }[];
  providerNotes?: string;
  recommendedActions?: string[];
}
```

Visual Requirements:

- Modal size: 600px width, auto height
- Table format for individual test components
- Color coding: Green for normal, yellow for borderline, red for abnormal
- Provider notes section at bottom
- Print/download functionality

Acceptance Criteria:

- Must only open for results with `status: 'normal'`
- Must show all test components with reference ranges
- Must provide clear interpretation of values
- Must include provider context when available

9.4 Contact Office Workflow (Abnormal Results)

Contact Modal:

```
typescript
```

```
interface ContactOfficeForm {  
  resultId: string;  
  contactMethod: 'phone' | 'message' | 'appointment';  
  urgency: 'routine' | 'urgent';  
  message?: string;  
  preferredContactTime?: string;  
}
```

Implementation:

- Phone: Show office number with click-to-call
- Message: Open secure messaging form
- Appointment: Link to appointment booking with pre-filled reason
- Auto-populate reason: "Regarding lab results from [date]"

Acceptance Criteria:

- Must clearly explain why contacting office is required
- Must provide multiple contact options
- Must pre-populate relevant context
- Must track patient communication for follow-up

10. Patient Appointment Management (/patient/appointments)

10.1 Appointments Dashboard

Upcoming Appointments Section:

typescript

```
interface UpcomingAppointment {  
  id: string;  
  date: string;  
  time: string;  
  provider: string;  
  type: string;  
  location: string;  
  status: 'confirmed' | 'needs_confirmation' | 'cancelled';  
  canReschedule: boolean;  
  canCancel: boolean;  
  remindersSent: number;  
  checkInAvailable: boolean;  
}
```

Visual Requirements:

- Card layout (similar to billing)
- Appointment cards show: Date/time, provider, type, location
- Action buttons: "Reschedule", "Cancel", "Check In" (if within window)
- Status indicators with appropriate colors
- "I'm Here" button appears 30 minutes before appointment

Check-In Functionality:

typescript

```
interface CheckInProcess {  
  appointmentId: string;  
  checkInTime: string;  
  insuranceVerified: boolean;  
  formsCompleted: string[];  
  estimatedWaitTime: number;  
}
```

Acceptance Criteria:

- "I'm Here" button only appears within check-in window (30 min before to 15 min after)
- Must verify insurance information
- Must confirm required forms are completed
- Must provide estimated wait time

- Must update status in clinic systems

10.2 Appointment Booking (/patient/appointments/new)

Booking Wizard Steps:

1. **Select Provider** - Filter by specialty, show availability
2. **Select Date/Time** - Calendar view with available slots
3. **Appointment Type** - Routine, follow-up, problem-focused
4. **Reason for Visit** - Text area with character limit
5. **Insurance Verification** - Confirm current insurance
6. **Confirmation** - Review and confirm details

Provider Selection:

```
typescript

interface ProviderOption {
  id: string;
  name: string;
  title: string;
  specialty: string;
  nextAvailableDate: string;
  image?: string;
  acceptingNewPatients: boolean;
}
```

Visual Requirements:

- Provider cards with photos and credentials
- Availability indicator (next available date)
- Filter by specialty dropdown
- "Not accepting new patients" disabled state

Calendar Interface:

- Month view with available dates highlighted
- Time slot selection for chosen date
- Different colors for different appointment types
- Unavailable slots grayed out

- Mobile-friendly date/time picker

Acceptance Criteria:

- Must check provider availability in real-time
- Must validate insurance coverage for selected provider
- Must prevent double-booking
- Must send confirmation email/SMS
- Must integrate with clinic scheduling system

10.3 Appointment History

Historical Appointment Display:

```
typescript

interface PastAppointment {
  id: string;
  date: string;
  provider: string;
  type: string;
  visitSummary?: string;
  followUpInstructions?: string;
  prescriptionsGiven?: Prescription[];
  nextAppointmentRecommended?: boolean;
}
```

Features:

- Sortable by date, provider, type
- Search functionality
- Visit summaries (when available)
- Links to related lab results or prescriptions
- "Schedule Follow-up" action for applicable visits

11. Patient Profile Management (/patient/profile)

11.1 Personal Information Section

```
typescript
```

```
interface PatientProfile {
  personalInfo: {
    firstName: string;
    lastName: string;
    dateOfBirth: string;
    phone: string;
    email: string;
    address: Address;
    emergencyContact: EmergencyContact;
  };
  medicalInfo: {
    allergies: Allergy[];
    medications: Medication[];
    medicalHistory: MedicalCondition[];
    familyHistory: FamilyHistory[];
  };
  preferences: {
    communicationMethod: 'email' | 'sms' | 'phone';
    appointmentReminders: boolean;
    labResultNotifications: boolean;
    marketingOptOut: boolean;
  };
}
```

11.2 Editable Form Implementation

Form Validation Rules:

- Required fields: Name, DOB, phone, email
- Email format validation
- Phone number formatting (auto-format as user types)
- Date validation for DOB
- Address validation with autocomplete

Security Considerations:

- Audit trail for all profile changes
- Verify identity for sensitive changes (DOB, SSN)
- Two-factor authentication for email/phone changes
- HIPAA logging for all access/modifications

12. Secure Messaging (/patient/messages)

12.1 Message Thread Interface

typescript

```
interface MessageThread {
  id: string;
  subject: string;
  participants: string[];
  lastMessage: Message;
  unreadCount: number;
  priority: 'low' | 'normal' | 'high' | 'urgent';
  category: 'general' | 'appointment' | 'lab_results' | 'billing' | 'prescription';
}

interface Message {
  id: string;
  threadId: string;
  from: string;
  to: string[];
  content: string;
  timestamp: string;
  attachments?: Attachment[];
  isRead: boolean;
  messageType: 'text' | 'system' | 'appointment_reminder' | 'lab_alert';
}
```

12.2 Message Composition

Features:

- Rich text editor with formatting options
- File attachment capability (PDFs, images only)
- Recipient selection (dropdown of care team)
- Subject line with auto-suggestions based on category
- Priority setting
- Read receipt requests

Security Requirements:

- End-to-end encryption for message content

- Audit logging for all message access
- Automatic message expiration (configurable)
- Attachment virus scanning
- File size limits (10MB per attachment)

12.3 Message Categories & Templates

Pre-defined Categories:

1. **General Questions** - Template: "I have a question about..."
2. **Appointment Requests** - Template: "I'd like to schedule..."
3. **Prescription Refills** - Template: "I need a refill for..."
4. **Lab Result Questions** - Template: "I have questions about my recent lab results..."
5. **Billing Inquiries** - Template: "I have a question about my bill..."

13. AI Patient Communication Assistant

13.1 Chat Interface Integration

Access Points:

1. **Floating Chat Button:** Bottom-right corner of all patient portal pages
2. **Contact Office Quick Action:** Links to chat interface
3. **Emergency Contact:** Available 24/7 with escalation protocols

Visual Requirements:

```
typescript

interface ChatInterface {
  position: 'floating' | 'fullscreen' | 'sidebar';
  theme: 'light' | 'dark';
  headerColor: string; // Practice brand color
  botAvatar: string; // Practice logo or generic AI icon
  humanAvatar: string; // Staff member photos when transferred
}
```

Chat Widget Styling:

- **Floating Button:** 60px diameter, practice brand color, chat icon
- **Chat Window:** 380px width × 500px height on desktop

- **Mobile:** Full screen overlay with close button
- **Header:** Practice name + "AI Assistant" or staff name when transferred
- **Status Indicator:** "AI Assistant" vs "Connected to [Staff Name]"

13.2 AI Assistant Capabilities & Limitations

Core Functionalities:

typescript

```
interface AIAssistantCapabilities {  
  // ALLOWED - Administrative/Informational  
  officeHours: boolean;  
  appointmentScheduling: boolean;  
  appointmentRescheduling: boolean;  
  appointmentCancellation: boolean;  
  billingInquiries: boolean;  
  insuranceQuestions: boolean;  
  prescriptionRefillRequests: boolean;  
  labResultStatus: boolean; // "Your results are ready" - not interpretation  
  officeDirections: boolean;  
  parkingInformation: boolean;  
  practicePolicy: boolean;  
  
  // FORBIDDEN - Medical/Clinical  
  medicalAdvice: false;  
  symptomDiagnosis: false;  
  medicationRecommendations: false;  
  labResultInterpretation: false;  
  emergencyTriage: false; // Must escalate immediately  
  dosageInstructions: false;  
  treatmentPlans: false;  
}
```

Medical Advice Prevention System:

typescript

```

interface MedicalTopicDetection {
  keywords: string[]; // "pain", "symptoms", "medication", "diagnosis", etc.
  phrases: string[]; // "what should I do", "is this normal", "should I be worried"
  confidenceThreshold: number; // 0.7 - if >70% confidence it's medical, escalate
  escalationTriggers: string[]; // "emergency", "urgent", "severe pain", "can't breathe"
}

const MEDICAL_KEYWORDS = [
  'pain', 'hurt', 'symptom', 'sick', 'fever', 'medication', 'pill', 'dose',
  'diagnosis', 'treatment', 'surgery', 'infection', 'allergy', 'reaction',
  'bleeding', 'swelling', 'rash', 'dizzy', 'nausea', 'vomiting', 'chest pain',
  'shortness of breath', 'emergency', 'urgent', 'severe', 'worried', 'scared'
];

const EMERGENCY_KEYWORDS = [
  'emergency', 'urgent', 'can\'t breathe', 'chest pain', 'severe pain',
  'bleeding heavily', 'unconscious', 'stroke', 'heart attack', 'suicide'
];

```

13.3 Office Hours vs After-Hours Behavior

During Office Hours (8 AM - 5 PM, Mon-Fri):

```

typescript

interface OfficeHoursMode {
  maxAIConversationTime: 5; // minutes before offering human transfer
  humanTransferAvailable: true;
  responseTime: 'immediate'; // <2 seconds
  escalationThreshold: 'low'; // Transfer quickly to humans
  availableStaff: StaffMember[];
}

```

Behavior:

- AI handles basic inquiries for max 5 minutes
- Proactively offers human transfer: "Would you like me to connect you with our front office staff?"
- Medical questions: Immediate transfer to clinical staff
- Complex billing: Transfer to billing specialist
- Urgent issues: Immediate escalation with alert to staff

After Office Hours (Nights, Weekends, Holidays):

typescript

```
interface AfterHoursMode {  
  maxAIConversationTime: 15; // minutes - more AI assistance when no staff  
  humanTransferAvailable: false;  
  responseTime: 'immediate';  
  escalationThreshold: 'high'; // Only for true emergencies  
  emergencyProtocol: EmergencyEscalationFlow;  
}
```

Behavior:

- AI provides more comprehensive non-medical assistance
- Emergency detection with immediate escalation protocol
- Appointment requests: Schedule for next business day callback
- Prescription refills: Submit request for next day processing
- Medical questions: "I can't provide medical advice. If this is urgent, please call [emergency number] or visit the ER."

13.4 Conversation Flow & Escalation Logic

Initial Greeting:

typescript

```
const getGreeting = (timeOfDay: 'office' | 'after_hours') => {  
  const baseGreeting = "Hi! I'm the AI assistant for [Practice Name]. ";  
  
  if (timeOfDay === 'office') {  
    return baseGreeting + "I can help with appointments, billing, and general questions. For medical concerns, I'll connect you to a doctor.";  
  } else {  
    return baseGreeting + "Our office is currently closed, but I can help with some basic questions and take messages for the next business day.";  
  }  
};
```

Escalation Decision Tree:

typescript

```
interface EscalationLogic {
  checkMedicalContent: (message: string) => {
    isMedical: boolean;
    isEmergency: boolean;
    confidence: number;
  };

  escalatelf: {
    emergencyKeywords: boolean;
    medicalAdviceRequested: boolean;
    userFrustration: boolean; // "this isn't helpful", "I need a human"
    complexBilling: boolean;
    conversationTooLong: boolean;
    aiConfusionDetected: boolean; // AI can't understand user
  };
}
```

Human Transfer Process:

```
typescript

interface TransferProcess {
  // During office hours
  office_hours: {
    step1: "Let me connect you with our staff member";
    step2: "Please hold while I find someone available";
    step3: "You're now connected with [Staff Name]";
    fallback: "All staff are busy. Would you like to leave a message?";
  };

  // After hours
  after_hours: {
    step1: "Our office is closed, but I can take a detailed message";
    step2: "A staff member will respond first thing tomorrow";
    step3: "For emergencies, please call [emergency number]";
  };
}
```

13.5 Message Taking & Follow-up System

After-Hours Message Collection:

```
typescript
```

```
interface AfterHoursMessage {
  patientId: string;
  timestamp: string;
  messageType: 'appointment_request' | 'billing_question' | 'prescription_refill' | 'general_inquiry' | 'urgent_callback';
  priority: 'low' | 'normal' | 'high' | 'urgent';
  patientMessage: string;
  aiSummary: string; // AI-generated summary of the conversation
  suggestedAction: string; // "Schedule callback", "Process refill", etc.
  preferredContactMethod: 'phone' | 'email' | 'portal_message';
  preferredContactTime: string;
}
```

Staff Notification System:

```
typescript

interface StaffNotification {
  messageId: string;
  priority: 'urgent' | 'high' | 'normal';
  patientName: string;
  messagePreview: string; // First 100 characters
  suggestedAction: string;
  deliveryMethod: 'email' | 'sms' | 'portal_alert' | 'all';
  escalationTime: string; // When to escalate if not responded to
}
```

13.6 Emergency Detection & Protocol

Emergency Response Flow:

```
typescript
```

```
interface EmergencyProtocol {
  detection: {
    keywords: string[];
    phrases: string[];
    sentiment: 'panic' | 'urgent' | 'distressed';
  };

  response: {
    immediate: "This sounds like a medical emergency. Please call 911 immediately or go to your nearest emergency room.";
    followUp: "I'm also alerting our on-call provider about your situation.";
    resources: {
      emergency: "911";
      poisonControl: "1-800-222-1222";
      suicideHotline: "988";
      nearestER: string; // Based on patient address
    };
  };

  escalation: {
    alertOnCallProvider: boolean;
    createUrgentTicket: boolean;
    requireFollowUp: boolean;
  };
}
```

13.7 Conversation Logging & Analytics

Data Collection:

typescript

```
interface ConversationLog {
  sessionId: string;
  patientId: string;
  startTime: string;
  endTime: string;
  messageCount: number;
  wasTransferredToHuman: boolean;
  transferReason: string;
  wasEmergency: boolean;
  satisfactionRating?: number; // 1-5 stars post-conversation
  resolutionStatus: 'resolved' | 'needs_follow_up' | 'escalated' | 'emergency';
  aiPerformanceMetrics: {
    accurateResponses: number;
    inappropriateResponses: number;
    medicalAdviceAttempts: number;
  };
}
```

AI Training Data:

```
typescript

interface TrainingDataPoint {
  userMessage: string;
  aiResponse: string;
  wasEscalated: boolean;
  escalationReason: string;
  humanRating: 'appropriate' | 'inappropriate' | 'needs_improvement';
  correctResponse?: string; // If AI response was wrong
}
```

13.8 Technical Implementation

Real-time Chat Architecture:

```
typescript
```

```
interface ChatArchitecture {  
  frontend: 'WebSocket connection for real-time messaging';  
  backend: 'Node.js with Socket.io for chat handling';  
  aiService: 'Google Vertex AI with custom medical content filters';  
  humanHandoff: 'Integration with staff messaging system';  
  emergencyAlerts: 'SMS/email notification system';  
}
```

Response Time Requirements:

- **AI Response:** <2 seconds
- **Human Transfer:** <30 seconds during office hours
- **Emergency Detection:** <1 second with immediate escalation
- **After-hours Message:** Confirmed receipt within 2 seconds

13.9 Compliance & Security

HIPAA Compliance:

- All conversations encrypted in transit and at rest
- Audit logging for all message access
- Patient consent for AI communication
- Data retention policies (7 years for medical, 3 years for administrative)
- Staff training on AI limitations and escalation protocols

Content Filtering:

```
typescript  
  
interface ContentFilter {  
  medicalAdviceDetection: boolean;  
  offensiveLanguageFilter: boolean;  
  PHISharingPrevention: boolean;  
  inappropriateRequestHandling: boolean;  
  spamDetection: boolean;  
}
```

Acceptance Criteria:

- AI must never provide medical advice under any circumstances

- Emergency situations must be detected and escalated within 1 second
- Human transfer must be available during office hours within 30 seconds
- All conversations must be logged for compliance and training
- Patient satisfaction rating must be >4.0/5.0 on average
- False medical advice detection rate must be <1%
- System uptime must be >99.9%

This comprehensive expansion covers the AI chatbot feature with all the safety protocols, escalation procedures, and technical requirements needed for a HIPAA-compliant patient communication system.